Notion API Documentation

FIELD	DETAILS
API Name	Notion
Version	2026-03-11
Base URL	https://api.notion.com/v1
Author	Gokul S Anand
Last Updated	April 23, 2026

Overview

Notion is a global platform developed by Notion.inc for documentation, project management, and knowledge bases. This guide explains the API endpoints for external developers building Notion integrations who are familiar with REST APIs but new to Notion's data model.

Base URL

https://api.notion.com/v1

All endpoints are relative to this base URL.

Authentication

To get an integration token:

Step 1: Create an integration

Go to https://www.notion.so/my-integrations > create integration > copy the secret_ token

Step 2: Add the integration to your page

Open any Notion page > Settings > Connections > select your integration

Format of the integration token: secret_xxxxxxxxx

Authorization: Bearer secret_xxxxx

> ⚠️ **Warning:** Never share your API key publicly. Keep it secure at all times.

Rate Limiting

The rate limit for incoming requests per integration is an average of three requests per second. Some bursts beyond the average rate are allowed.

Request Format

HEADER	VALUE
Content-Type	application/json
Authorization	Bearer YOUR_KEY
Notion-Version	2026-03-11

Response Format

All responses return JSON:

```
{
    "object": "page",
    "id": "34051562-df03-81d0-b27f-e1622a0bf74f",
    "created_time": "2026-04-12T12:36:00.000Z",
    "last_edited_time": "2026-04-12T12:36:00.000Z",
    "parent": {
        "type": "page_id",
        "page_id": "34051562-df03-806a-b9d7-cd40439aef35"
    },
    "in_trash": false,
    "is_archived": false,
    "url": "https://www.notion.so/new-notion-
34051562df0381d0b27fe1622a0bf74f"
}
```

Endpoints

1. Create a Page

Creates a new page inside the parent pages with properties.

Endpoint:

POST /pages

Headers:

HEADER	VALUE
Authorization	Bearer secret_xxxxxxxxx
Content-Type	application/json
Notion-Version	2026-03-11

Request:

```
{
  "parent": {
    "page_id": "34051562df03806ab9d7cd40439aef35"
  },
  "properties": {
    "title": {
      "title": [{ "text": { "content": "new notion" } }]
    }
  }
}
```

Request Fields:

FIELDS	TYPE	REQUIRED	DESCRIPTION
`parent`	object	Yes	Information about the page's parent.
`page_id`	string	Yes	Unique identifier of the parent page.
`properties`	object	Yes	Property values of this page.
`title`	object	Yes	A type object that contains data specific to the page being created
`text`	array	Yes	Array of text objects containing the page content
`content`	string	Yes	The content of title of the creating page

Response:

```
{
    "object": "page",
    "id": "34051562-df03-81d0-b27f-e1622a0bf74f",
    "created_time": "2026-04-12T12:36:00.000Z",
```

```
    "last_edited_time": "2026-04-12T12:36:00.000Z",
    "created_by": {
        "object": "user",
        "id": "33f51562-df03-81b1-bc3a-0027d86fa2da"
    },
    "last_edited_by": {
        "object": "user",
        "id": "33f51562-df03-81b1-bc3a-0027d86fa2da"
    },
    "cover": null,
    "icon": null,
    "parent": {
        "type": "page_id",
        "page_id": "34051562-df03-806a-b9d7-cd40439aef35"
    },
    "in_trash": false,
    "is_archived": false,
    "is_locked": false,
    "properties": {
        "title": {
            "id": "title",
            "type": "title",
            "title": [
                {
                    "type": "text",
                    "text": {
                        "content": "new notion",
                        "link": null
                    },
                    "annotations": {
                        "bold": false,
                        "italic": false,
                        "strikethrough": false,
                        "underline": false,
                        "code": false,
                        "color": "default"
                    },
                    "plain_text": "new notion",
                    "href": null
                }
            ]
        }
    },
    "url": "https://www.notion.so/new-notion-
34051562df0381d0b27fe1622a0bf74f",
    "public_url": null,
    "archived": false
}
```

Response Fields:

FIELD	TYPE	DESCRIPTION
`object`	string	Always `"page"` for page responses
`id`	string	Unique ID of the newly created page
`created_time`	string	ISO 8601 timestamp of page creation
`last_edited_time`	string	ISO 8601 timestamp of last edit
`parent`	object	Contains the parent page or database ID
`url`	string	Direct URL to the page in Notion
`in_trash`	boolean	Whether the page is deleted
`is_archived`	boolean	Whether the page is archived
`properties`	object	Contains page title and other properties

Status Codes:

CODE	MEANING
200	Page created successfully
400	Bad request — missing or invalid fields
401	Unauthorized — invalid or missing token
404	Parent page not found
500	Internal server error

---

2. Fetch a Page by ID

Retrieve a page by its ID.

Endpoint:

GET /pages/:id

Path Parameters:

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`page_id`	string	Yes	Unique ID of the page

Headers:

HEADER	VALUE
Authorization	Bearer secret_xxxxxxxxx
Content-Type	application/json
Notion-Version	2026-03-11

Response:


```
{
    "object": "page",
    "id": "34051562-df03-806a-b9d7-cd40439aef35",
    "created_time": "2026-04-12T11:55:00.000Z",
    "last_edited_time": "2026-04-13T06:05:00.000Z",
    "created_by": {
        "object": "user",
        "id": "33ed872b-594c-81ef-a03c-00026e2d4efd"
    },
    "last_edited_by": {
        "object": "user",
        "id": "33f51562-df03-81b1-bc3a-0027d86fa2da"
    },
    "cover": {
        "type": "external",
        "external": {
            "url": "https://www.notion.so/images/page-
cover/nationalMuseumOfAsianArt_landscapeWithGibbonsAndCranes.jpg"
        }
    },
    "icon": null,
    "parent": {
        "type": "workspace",
        "workspace": true
    },
    "in_trash": false,
    "is_archived": false,
    "is_locked": false,
    "properties": {
        "title": {
            "id": "title",
            "type": "title",
            "title": [
                {
                    "type": "text",
                    "text": {
                        "content": "Notion the Office App",
                        "link": null
                    },
```

```
                        "annotations": {
                            "bold": false,
                            "italic": false,
                            "strikethrough": false,
                            "underline": false,
                            "code": false,
                            "color": "default"
                        },
                        "plain_text": "Notion the Office App",
                        "href": null
                    }
                ]
            }
        },
        "url": "https://www.notion.so/Notion-the-Office-App-
34051562df03806ab9d7cd40439aef35",
        "public_url": null,
        "archived": false
    }
```


Response Fields:

FIELD	TYPE	DESCRIPTION
`object`	string	Always `"page"` for page responses
`id`	string	Unique ID of the retrieved page
`created_time`	string	ISO 8601 timestamp of page creation
`last_edited_time`	string	ISO 8601 timestamp of last edit
`created_by`	object	User who created the page
`last_edited_by`	object	User who last edited the page
`cover`	object	Cover image of the page, null if not set
`parent`	object	Parent container — workspace, page, or database
`in_trash`	boolean	Whether the page is deleted
`is_archived`	boolean	Whether the page is archived
`is_locked`	boolean	Whether the page is locked from editing
`properties`	object	Contains page title and other properties
`url`	string	Direct URL to the page in Notion
`public_url`	string	Public URL if page is shared, null otherwise

Status Codes:

CODE	MEANING
200	Page retrieved successfully
401	Unauthorized — invalid or missing token
404	Page not found
500	Internal server error

---

3. Update Page Properties

Updates the properties of an existing page using its unique page ID.

Endpoint:

PATCH /pages/:id

Path parameters:

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`page_id`	string	Yes	Unique ID of the page

Headers:

HEADER	VALUE
Authorization	Bearer secret_xxxxxxxxx
Content-Type	application/json
Notion-Version	2026-03-11

Request:

```
{
  "properties": {
    "title": {
      "title": [{ "text": { "content": "gokul" } }]
    }
  }
}
```

Request Fields:

FIELDS	TYPE	REQUIRED	DESCRIPTION
`properties`	object	Yes	Property values of this page.
`title`	string	Yes	A type object that contains data specific to the page being created
`text`	string	Yes	The type of the property in the page object.
`content`	string	Yes	Changed name of the page

Response:


```
{
    "object": "page",
    "id": "34051562-df03-806a-b9d7-cd40439aef35",
    "created_time": "2026-04-12T11:55:00.000Z",
    "last_edited_time": "2026-04-13T07:16:00.000Z",
    "created_by": {
        "object": "user",
        "id": "33ed872b-594c-81ef-a03c-00026e2d4efd"
    },
    "last_edited_by": {
        "object": "user",
        "id": "33f51562-df03-81b1-bc3a-0027d86fa2da"
    },
    "cover": {
        "type": "external",
        "external": {
            "url": "https://www.notion.so/images/page-
cover/nationalMuseumOfAsianArt_landscapeWithGibbonsAndCranes.jpg"
        }
    },
    "icon": null,
    "parent": {
        "type": "workspace",
        "workspace": true
    },
    "in_trash": false,
    "is_archived": false,
    "is_locked": false,
    "properties": {
        "title": {
            "id": "title",
            "type": "title",
            "title": [
                {
```

```
                        "type": "text",
                        "text": {
                            "content": "gokul",
                            "link": null
                        },
                        "annotations": {
                            "bold": false,
                            "italic": false,
                            "strikethrough": false,
                            "underline": false,
                            "code": false,
                            "color": "default"
                        },
                        "plain_text": "gokul",
                        "href": null
                    }
                ]
            }
        },
        "url": "https://www.notion.so/gokul-
34051562df03806ab9d7cd40439aef35",
        "public_url": null,
        "archived": false,
    }
```


Response Fields:

FIELD	TYPE	DESCRIPTION
`object`	string	Always `"page"` for page responses
`id`	string	Unique ID of the retrieved page
`created_time`	string	ISO 8601 timestamp of page creation
`last_edited_time`	string	ISO 8601 timestamp of last edit
`created_by`	object	User who created the page
`last_edited_by`	object	User who last edited the page
`cover`	object	Cover image of the page, null if none
`parent`	object	Parent container — workspace, page, or database
`in_trash`	boolean	Whether the page is deleted
`is_archived`	boolean	Whether the page is archived
`is_locked`	boolean	Whether the page is locked from editing

FIELD	TYPE	DESCRIPTION
`properties`	object	Contains page title and other properties
`url`	string	Direct URL to the page in Notion
`public_url`	string	Public URL if page is shared, null if private

Status Codes:

CODE	MEANING
200	successfully updated page properties
400	Bad request — missing fields
401	Unauthorized
500	Internal server error

---

4. Query a Database

Returns all pages in a database. Supports filters, sorts, and pagination.

Endpoint:

POST https://api.notion.com/v1/databases/{database_id}/query

Path Parameters:

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`database_id`	string	Yes	Unique ID of the database to query
`query`	string	Yes	collect details about the database

Headers:

HEADER	VALUE
Authorization	Bearer secret_xxxxxxxxx
Content-Type	application/json
Notion-Version	2026-03-11

Request:

Send empty body to return all rows:

```
{}
```

Response:


```
{
    "object": "list",
    "results": [
        {
            "object": "page",
            "id": "34151562-df03-805d-9fbd-ce4dbc543edd",
            "created_time": "2026-04-13T12:08:00.000Z",
            "last_edited_time": "2026-04-13T12:17:00.000Z",
            "parent": {
                "type": "database_id",
                "database_id": "34151562-df03-80c3-a20b-fed75ebccced"
            },
            "properties": {
                "Column 2": { "type": "rich_text", "plain_text": "YEAR
JOINED" },
                "Column 1": { "type": "rich_text", "plain_text": "AGE"
},
                "Title": { "type": "title", "plain_text": "NAME" }
            }
        }
    ],
    "next_cursor": null,
    "has_more": false,
    "type": "page_or_database"
}
```


With Filter:

```
{
  "filter": {
    "property": "Column 1",
    "rich_text": {
      "equals": "23"
    }
  }
}
```

Response:


```
{
    "object": "list",
    "results": [
        {
            "object": "page",
            "id": "34151562-df03-80ca-8cfa-e652f18f2af5",
            "created_time": "2026-04-13T12:08:00.000Z",
            "last_edited_time": "2026-04-13T12:17:00.000Z",
            "parent": {
                "type": "database_id",
                "database_id": "34151562-df03-80c3-a20b-fed75ebccced"
            },
            "properties": {
                "Column 2": { "type": "rich_text", "plain_text": "2026"
},
                "Column 1": { "type": "rich_text", "plain_text": "23" },
                "Title": { "type": "title", "plain_text": "gokul" }
            }
        }
    ],
    "next_cursor": null,
    "has_more": false,
    "type": "page_or_database"
}
```


With Sort:

```
{
  "sorts": [
    {
      "property": "Column 1",
      "direction": "ascending"
    }
  ]
}
```

Response:


```
{
    "object": "list",
    "results": [
```

```
            {
                "object": "page",
                "id": "34151562-df03-806f-aabe-f90b016713c5",
                "created_time": "2026-04-13T12:09:00.000Z",
                "parent": {
                    "type": "database_id",
                    "database_id": "34151562-df03-80c3-a20b-fed75ebccced"
                },
                "properties": {
                    "Column 2": { "type": "rich_text", "plain_text": "2024"
},
                    "Column 1": { "type": "rich_text", "plain_text": "22" },
                    "Title": { "type": "title", "plain_text": "prime" }
                }
            },
            {
                "object": "page",
                "id": "34151562-df03-80ca-8cfa-e652f18f2af5",
                "created_time": "2026-04-13T12:08:00.000Z",
                "parent": {
                    "type": "database_id",
                    "database_id": "34151562-df03-80c3-a20b-fed75ebccced"
                },
                "properties": {
                    "Column 2": { "type": "rich_text", "plain_text": "2026"
},
                    "Column 1": { "type": "rich_text", "plain_text": "23" },
                    "Title": { "type": "title", "plain_text": "gokul" }
                }
            },
            {
                "object": "page",
                "id": "34151562-df03-80b2-95e4-c96d3490748c",
                "created_time": "2026-04-13T12:08:00.000Z",
                "parent": {
                    "type": "database_id",
                    "database_id": "34151562-df03-80c3-a20b-fed75ebccced"
                },
                "properties": {
                    "Column 2": { "type": "rich_text", "plain_text": "2025"
},
                    "Column 1": { "type": "rich_text", "plain_text": "34" },
                    "Title": { "type": "title", "plain_text": "manohar" }
                }
            }
        ],
        "next_cursor": null,
        "has_more": false,
        "type": "page_or_database"
}
```

Response Fields:

FIELD	TYPE	DESCRIPTION
`object`	string	Always `"list"` for query responses
`results`	array	Array of page objects matching the query
`results[].object`	string	Always `"page"` for each result
`results[].id`	string	Unique ID of the page
`results[].created_time`	string	ISO 8601 timestamp of page creation
`results[].last_edited_time`	string	ISO 8601 timestamp of last edit
`results[].parent`	object	Parent database containing the page
`results[].properties`	object	Column values of the database row
`results[].url`	string	Direct URL to the page in Notion
`results[].in_trash`	boolean	Whether the page is deleted
`next_cursor`	string	Cursor for next page of results, null if none
`has_more`	boolean	Whether more results exist beyond this page
`type`	string	Always `"page_or_database"` for query responses

> **Pagination note:** When `has_more` is `true`, pass the `next_cursor` value in your next request body as `"start_cursor"` to fetch the next page of results.

Status Codes:

CODE	MEANING
200	successful
400	Bad request
404	Resource not found
500	Internal server error

5. Retrieve Database Schema

Get the database schema.

Endpoint:

GET /databases/:id

Path Parameters:

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`database_id`	string	Yes	Unique ID of the database

Headers:

HEADER	VALUE
Authorization	Bearer secret_xxxxxxxxx
Content-Type	application/json
Notion-Version	2026-03-11

Response:


```
{
    "object": "database",
    "id": "34151562-df03-80c3-a20b-fed75ebccced",
    "cover": null,
    "icon": null,
    "created_time": "2026-04-13T12:08:00.000Z",
    "created_by": {
        "object": "user",
        "id": "33ed872b-594c-81ef-a03c-00026e2d4efd"
    },
    "last_edited_by": {
        "object": "user",
        "id": "33ed872b-594c-81ef-a03c-00026e2d4efd"
    },
    "last_edited_time": "2026-04-13T12:17:00.000Z",
    "title": [],
    "description": [],
    "is_inline": true,
    "properties": {
        "Column 2": {
            "id": "%5Cqk%5E",
            "name": "Column 2",
            "description": null,
            "type": "rich_text",
            "rich_text": {}
        },
```

```
            "Column 1": {
                "id": "ef%3AI",
                "name": "Column 1",
                "description": null,
                "type": "rich_text",
                "rich_text": {}
            },
            "Title": {
                "id": "title",
                "name": "Title",
                "description": null,
                "type": "title",
                "title": {}
            }
        },
        "parent": {
            "type": "page_id",
            "page_id": "34151562-df03-8069-a254-d0c0b7e4c8e5"
        },
        "url": "https://www.notion.so/34151562df0380c3a20bfed75ebccced",
        "public_url": null,
        "in_trash": false,
        "archived": false,
```


Response Fields:

FIELD	TYPE	DESCRIPTION
`object`	string	Always `"database"` for database responses
`id`	string	Unique ID of the database
`created_time`	string	ISO 8601 timestamp of database creation
`last_edited_time`	string	ISO 8601 timestamp of last edit
`created_by`	object	User who created the database
`last_edited_by`	object	User who last edited the database
`title`	array	Title of the database
`description`	array	Description of the database
`properties`	object	Schema of all database columns
`parent`	object	Parent page containing the database
`is_inline`	boolean	Whether database is inline or full page
`url`	string	Direct URL to the database in Notion

FIELD	TYPE	DESCRIPTION
`in_trash`	boolean	Whether the database is deleted
`archived`	boolean	Whether the database is archived

Status Codes:

CODE	MEANING
200	Retrieved database schema
404	Resource not found
500	Internal server error

---

6. Retrieve child blocks of a page

Get child blocks details from a page.

Endpoint:

GET /blocks/:id/children

Path Parameters:

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`block_id`	string	Yes	Unique ID of the block
`children`	array	yes	blocks inside a parent block

Headers:

HEADER	VALUE
Authorization	Bearer secret_xxxxxxxxx
Content-Type	application/json
Notion-Version	2026-03-11

Response:

```
{
    "object": "list",
    "results": [
```

```
        {
            "object": "block",
            "id": "34051562-df03-80c0-bc43-fce363bee5bf",
            "parent": {
                "type": "page_id",
                "page_id": "34051562-df03-806a-b9d7-cd40439aef35"
            },
            "created_time": "2026-04-12T11:56:00.000Z",
            "last_edited_time": "2026-04-12T11:59:00.000Z",
            "created_by": {
                "object": "user",
                "id": "33ed872b-594c-81ef-a03c-00026e2d4efd"
            },
            "last_edited_by": {
                "object": "user",
                "id": "33ed872b-594c-81ef-a03c-00026e2d4efd"
            },
            "has_children": false,
            "in_trash": false,
            "type": "paragraph",
            "paragraph": {
                "rich_text": [
                    {
                        "type": "text",
                        "text": {
                            "content": "Notion is used by 30 million
people globally for documentation, project management, and knowledge
bases. We expose a public API allowing developers to build integrations
— automating page creation, database queries, and content updates. Our
current API docs are written by engineers and lack clarity for
developers new to our platform. We need a Technical Writer to rebuild
the core API reference.",
                            "link": null
                        },
                        "annotations": {
                            "bold": false,
                            "italic": false,
                            "strikethrough": false,
                            "underline": false,
                            "code": false,
                            "color": "default"
                        },
                        "plain_text": "Notion is used by 30 million
people globally for documentation, project management, and knowledge
bases. We expose a public API allowing developers to build integrations
— automating page creation, database queries, and content updates. Our
current API docs are written by engineers and lack clarity for
developers new to our platform. We need a Technical Writer to rebuild
the core API reference.",
                        "href": null
                    }
```

```
                ],
                "icon": null,
                "color": "default"
            },
            "archived": false
        },
        {
            "object": "block",
            "id": "34051562-df03-81d0-b27f-e1622a0bf74f",
            "parent": {
                "type": "page_id",
                "page_id": "34051562-df03-806a-b9d7-cd40439aef35"
            },
            "created_time": "2026-04-12T12:36:00.000Z",
            "last_edited_time": "2026-04-13T18:19:00.000Z",
            "created_by": {
                "object": "user",
                "id": "33f51562-df03-81b1-bc3a-0027d86fa2da"
            },
            "last_edited_by": {
                "object": "user",
                "id": "33ed872b-594c-81ef-a03c-00026e2d4efd"
            },
            "has_children": false,
            "in_trash": false,
            "type": "child_page",
            "child_page": {
                "title": "Brand"
            },
            "archived": false
        },
        {
            "object": "block",
            "id": "34151562-df03-80f6-a134-d0225736708b",
            "parent": {
                "type": "page_id",
                "page_id": "34051562-df03-806a-b9d7-cd40439aef35"
            },
            "created_time": "2026-04-13T10:55:00.000Z",
            "last_edited_time": "2026-04-13T10:55:00.000Z",
            "created_by": {
                "object": "user",
                "id": "33ed872b-594c-81ef-a03c-00026e2d4efd"
            },
            "last_edited_by": {
                "object": "user",
                "id": "33ed872b-594c-81ef-a03c-00026e2d4efd"
            },
            "has_children": false,
            "in_trash": false,
            "type": "paragraph",
```

```
            "paragraph": {
                "rich_text": [],
                "icon": null,
                "color": "default"
            },
            "archived": false
        },
        {
            "object": "block",
            "id": "34151562-df03-8146-92d4-cf446fbd43a5",
            "parent": {
                "type": "page_id",
                "page_id": "34051562-df03-806a-b9d7-cd40439aef35"
            },
            "created_time": "2026-04-13T06:05:00.000Z",
            "last_edited_time": "2026-04-13T06:05:00.000Z",
            "created_by": {
                "object": "user",
                "id": "33f51562-df03-81b1-bc3a-0027d86fa2da"
            },
            "last_edited_by": {
                "object": "user",
                "id": "33f51562-df03-81b1-bc3a-0027d86fa2da"
            },
            "has_children": false,
            "in_trash": false,
            "type": "child_page",
            "child_page": {
                "title": "new notion"
            },
            "archived": false
        },
        {
            "object": "block",
            "id": "34151562-df03-8157-ac8f-dd6065bdb7a6",
            "parent": {
                "type": "page_id",
                "page_id": "34051562-df03-806a-b9d7-cd40439aef35"
            },
            "created_time": "2026-04-13T08:28:00.000Z",
            "last_edited_time": "2026-04-13T08:28:00.000Z",
            "created_by": {
                "object": "user",
                "id": "33f51562-df03-81b1-bc3a-0027d86fa2da"
            },
            "last_edited_by": {
                "object": "user",
                "id": "33f51562-df03-81b1-bc3a-0027d86fa2da"
            },
            "has_children": false,
            "in_trash": false,
```

```
            "type": "child_page",
            "child_page": {
                "title": "new notion"
            },
            "archived": false
        },
        {
            "object": "block",
            "id": "34151562-df03-81a6-a2c3-c82b3003edd8",
            "parent": {
                "type": "page_id",
                "page_id": "34051562-df03-806a-b9d7-cd40439aef35"
            },
            "created_time": "2026-04-13T08:28:00.000Z",
            "last_edited_time": "2026-04-13T08:28:00.000Z",
            "created_by": {
                "object": "user",
                "id": "33f51562-df03-81b1-bc3a-0027d86fa2da"
            },
            "last_edited_by": {
                "object": "user",
                "id": "33f51562-df03-81b1-bc3a-0027d86fa2da"
            },
            "has_children": false,
            "in_trash": false,
            "type": "child_page",
            "child_page": {
                "title": "new notion"
            },
            "archived": false
        },
        {
            "object": "block",
            "id": "34151562-df03-8006-bebe-e7407547a187",
            "parent": {
                "type": "page_id",
                "page_id": "34051562-df03-806a-b9d7-cd40439aef35"
            },
            "created_time": "2026-04-13T10:56:00.000Z",
            "last_edited_time": "2026-04-13T10:56:00.000Z",
            "created_by": {
                "object": "user",
                "id": "33ed872b-594c-81ef-a03c-00026e2d4efd"
            },
            "last_edited_by": {
                "object": "user",
                "id": "33ed872b-594c-81ef-a03c-00026e2d4efd"
            },
            "has_children": false,
            "in_trash": false,
            "type": "paragraph",
```

```
                "paragraph": {
                    "rich_text": [],
                    "icon": null,
                    "color": "default"
                },
                "archived": false
            }
        ],
        "next_cursor": null,
        "has_more": false,
        "type": "block",
        "block": {},
    }
```

Response Fields:

FIELD	TYPE	DESCRIPTION
`object`	string	Always `"list"` for block list responses
`results`	array	Array of block objects inside the page
`results[].object`	string	Always `"block"` for each result
`results[].id`	string	Unique ID of the block
`results[].parent`	object	Parent page containing the block
`results[].created_time`	string	ISO 8601 timestamp of block creation
`results[].last_edited_time`	string	ISO 8601 timestamp of last edit
`results[].type`	string	Block type — `"paragraph"`, `"child_page"`, etc.
`results[].has_children`	boolean	Whether the block contains nested blocks
`results[].in_trash`	boolean	Whether the block is deleted
`results[].paragraph`	object	Paragraph content, present when type is `"paragraph"`
`results[].child_page`	object	Child page title, present when type is `"child_page"`
`next_cursor`	string	Cursor for next page of results, null if none
`has_more`	boolean	Whether more blocks exist beyond this page
`type`	string	Always `"block"` for block list responses

Status Codes:

CODE	MEANING
200	Child blocks retrieved successfully
404	Resource not found
500	Internal server error

---

7. Update a Block

Update a block's content.

Endpoint:

PATCH /blocks/:id

Path Parameters:

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`block_id`	string	Yes	Unique ID of the block

Headers:

HEADER	VALUE
Authorization	Bearer secret_xxxxxxxxx
Content-Type	application/json
Notion-Version	2026-03-11

Response:


```
{
    "object": "block",
    "id": "34051562-df03-80c0-bc43-fce363bee5bf",
    "parent": {
        "type": "page_id",
        "page_id": "34051562-df03-806a-b9d7-cd40439aef35"
    },
    "created_time": "2026-04-12T11:56:00.000Z",
    "last_edited_time": "2026-04-13T18:56:00.000Z",
    "created_by": {
        "object": "user",
```

```
            "id": "33ed872b-594c-81ef-a03c-00026e2d4efd"
        },
        "last_edited_by": {
            "object": "user",
            "id": "33f51562-df03-81b1-bc3a-0027d86fa2da"
        },
        "has_children": false,
        "in_trash": false,
        "type": "paragraph",
        "paragraph": {
            "rich_text": [
                {
                    "type": "text",
                    "text": {
                        "content": "make the title full bold",
                        "link": null
                    },
                    "annotations": {
                        "bold": false,
                        "italic": false,
                        "strikethrough": false,
                        "underline": false,
                        "code": false,
                        "color": "default"
                    },
                    "plain_text": "make the title full bold",
                    "href": null
                }
            ],
            "icon": null,
            "color": "default"
        },
        "archived": false
    }
```


Response Fields:

FIELD	TYPE	DESCRIPTION
`object`	string	Always `"block"` for block responses
`id`	string	Unique ID of the updated block
`parent`	object	Parent page containing the block
`created_time`	string	ISO 8601 timestamp of block creation
`last_edited_time`	string	ISO 8601 timestamp of last edit
`created_by`	object	User who created the block

FIELD	TYPE	DESCRIPTION
`last_edited_by`	object	User who last edited the block
`has_children`	boolean	Whether the block contains nested blocks
`in_trash`	boolean	Whether the block is deleted
`type`	string	Block type — always `"paragraph"` for paragraph blocks
`paragraph`	object	Contains the updated paragraph content
`paragraph.rich_text`	array	Array of text objects with content and annotations
`paragraph.color`	string	Text color — `"default"` unless changed
`archived`	boolean	Whether the block is archived

Status Codes:

CODE	MEANING
200	Block update success
404	Resource not found
500	Internal server error

---

8. Delete a block

Remove a block from the work

Endpoint:

DELETE /blocks/:id

Path Parameters:

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`block_id`	string	Yes	Unique ID of the block

Headers:

HEADER	VALUE
Authorization	Bearer secret_xxxxxxxxx
Content-Type	application/json

HEADER	VALUE
Notion-Version	2026-03-11

Response:


```
{
    "object": "block",
    "id": "34051562-df03-80c0-bc43-fce363bee5bf",
    "parent": {
        "type": "page_id",
        "page_id": "34051562-df03-806a-b9d7-cd40439aef35"
    },
    "created_time": "2026-04-12T11:56:00.000Z",
    "last_edited_time": "2026-04-14T06:32:00.000Z",
    "created_by": {
        "object": "user",
        "id": "33ed872b-594c-81ef-a03c-00026e2d4efd"
    },
    "last_edited_by": {
        "object": "user",
        "id": "33f51562-df03-81b1-bc3a-0027d86fa2da"
    },
    "has_children": false,
    "in_trash": true,
    "type": "paragraph",
    "paragraph": {
        "rich_text": [
            {
                "type": "text",
                "text": {
                    "content": "make the title full bold",
                    "link": null
                },
                "annotations": {
                    "bold": false,
                    "italic": false,
                    "strikethrough": false,
                    "underline": false,
                    "code": false,
                    "color": "default"
                },
                "plain_text": "make the title full bold",
                "href": null
            }
        ],
        "icon": null,
        "color": "default"
    },
```

```
    "archived": true
}
```

Response Fields:

FIELD	TYPE	DESCRIPTION
`object`	string	Always `"block"` for block responses
`id`	string	Unique ID of the deleted block
`parent`	object	Parent page that contained the block
`created_time`	string	ISO 8601 timestamp of block creation
`last_edited_time`	string	ISO 8601 timestamp when block was deleted
`created_by`	object	User who created the block
`last_edited_by`	object	User who deleted the block
`has_children`	boolean	Whether the block had nested blocks
`in_trash`	boolean	Always `true` after deletion
`type`	string	Block type before deletion
`paragraph`	object	Original paragraph content before deletion
`archived`	boolean	Always `true` after deletion

> Notion does not permanently delete blocks immediately. Deleted blocks have `in_trash: true` and `archived: true` and can be restored from Notion's trash.

Status Codes:

CODE	MEANING
200	Successfully deleted the block
404	Resource not found
500	Internal server error

Error Reference

CODE	ERROR	CAUSE	FIX
400	Invalid request	Missing or invalid data	Check request body

CODE	ERROR	CAUSE	FIX
401	Unauthorized token	Invalid or missing token	Check token
403	restricted resource	Given the bearer token used, the client doesn't have permission to perform this operation	API token does not have access to this resource.
404	Object Not Found	Resource does not exist	Verify endpoint and token
429	Too Many Requests	Rate limit exceeded	Wait and retry
500	Internal Server Error	Server-side issue	Retry or contact support

---

Code Examples Appendix

1. POST /pages — Create a Page

JavaScript:

```
fetch('https://api.notion.com/v1/pages', {
  method: 'POST',
  headers: {
    'Authorization': 'Bearer secret_xxxxxxxxx',
    'Content-Type': 'application/json',
    'Notion-Version': '2026-03-11'
  },
  body: JSON.stringify({
    parent: { page_id: '34051562df03806ab9d7cd40439aef35' },
    properties: {
      title: {
        title: [{ text: { content: 'new notion' } }]
      }
    }
  })
})
.then(response => response.json())
.then(data => console.log(data));
```

Python:

```python
import requests

headers = {
    'Authorization': 'Bearer secret_xxxxxxxxx',
    'Content-Type': 'application/json',
    'Notion-Version': '2026-03-11'
}

body = {
    'parent': { 'page_id': '34051562df03806ab9d7cd40439aef35' },
    'properties': {
        'title': {
            'title': [{ 'text': { 'content': 'new notion' } }]
        }
    }
}

response = requests.post(
    'https://api.notion.com/v1/pages',
    headers=headers,
    json=body
)

print(response.json())
```

2. GET /pages/:id — Retrieve a Page

JavaScript:

```javascript
fetch('https://api.notion.com/v1/pages/34051562df03806ab9d7cd40439aef35'
, {
  method: 'GET',
  headers: {
    'Authorization': 'Bearer secret_xxxxxxxxx',
    'Content-Type': 'application/json',
    'Notion-Version': '2026-03-11'
  }
})
.then(response => response.json())
.then(data => console.log(data));
```

Python:

```python
import requests

headers = {
    'Authorization': 'Bearer secret_xxxxxxxxx',
    'Content-Type': 'application/json',
    'Notion-Version': '2026-03-11'
}

response = requests.get(
    'https://api.notion.com/v1/pages/34051562df03806ab9d7cd40439aef35',
    headers=headers
)

print(response.json())
```

3. PATCH /pages/:id — Update a Page

JavaScript:

```javascript
fetch('https://api.notion.com/v1/pages/34051562df03806ab9d7cd40439aef35'
, {
  method: 'PATCH',
  headers: {
    'Authorization': 'Bearer secret_xxxxxxxxx',
    'Content-Type': 'application/json',
    'Notion-Version': '2026-03-11'
  },
  body: JSON.stringify({
    properties: {
      title: {
        title: [{ text: { content: 'gokul' } }]
      }
    }
  })
})
.then(response => response.json())
.then(data => console.log(data));
```

Python:

```python
import requests

headers = {
```

```
    'Authorization': 'Bearer secret_xxxxxxxxx',
    'Content-Type': 'application/json',
    'Notion-Version': '2026-03-11'
}

body = {
    'properties': {
        'title': {
            'title': [{ 'text': { 'content': 'gokul' } }]
        }
    }
}

response = requests.patch(
    'https://api.notion.com/v1/pages/34051562df03806ab9d7cd40439aef35',
    headers=headers,
    json=body
)

print(response.json())
```

4. POST /databases/:id/query — Query a Database

JavaScript:

```
fetch('https://api.notion.com/v1/databases/34151562df0380c3a20bfed75ebcc
ced/query', {
  method: 'POST',
  headers: {
    'Authorization': 'Bearer secret_xxxxxxxxx',
    'Content-Type': 'application/json',
    'Notion-Version': '2026-03-11'
  },
  body: JSON.stringify({
    filter: {
      property: 'Column 1',
      rich_text: { equals: '23' }
    }
  })
})
.then(response => response.json())
.then(data => console.log(data));
```

Python:

```python
import requests

headers = {
    'Authorization': 'Bearer secret_xxxxxxxxx',
    'Content-Type': 'application/json',
    'Notion-Version': '2026-03-11'
}

body = {
    'filter': {
        'property': 'Column 1',
        'rich_text': { 'equals': '23' }
    }
}

response = requests.post(

'https://api.notion.com/v1/databases/34151562df0380c3a20bfed75ebccced/qu
ery',
    headers=headers,
    json=body
)

print(response.json())
```

5. GET /databases/:id — Retrieve a Database

JavaScript:

```javascript
fetch('https://api.notion.com/v1/databases/34151562df0380c3a20bfed75ebcc
ced', {
  method: 'GET',
  headers: {
    'Authorization': 'Bearer secret_xxxxxxxxx',
    'Content-Type': 'application/json',
    'Notion-Version': '2026-03-11'
  }
})
.then(response => response.json())
.then(data => console.log(data));
```

Python:

```python
import requests

headers = {
    'Authorization': 'Bearer secret_xxxxxxxxx',
    'Content-Type': 'application/json',
    'Notion-Version': '2026-03-11'
}

response = requests.get(

'https://api.notion.com/v1/databases/34151562df0380c3a20bfed75ebccced',
    headers=headers
)

print(response.json())
```

6. GET /blocks/:id/children — Retrieve Child Blocks

JavaScript:

```javascript
fetch('https://api.notion.com/v1/blocks/34051562df03806ab9d7cd40439aef35
/children', {
  method: 'GET',
  headers: {
    'Authorization': 'Bearer secret_xxxxxxxxx',
    'Content-Type': 'application/json',
    'Notion-Version': '2026-03-11'
  }
})
.then(response => response.json())
.then(data => console.log(data));
```

Python:

```python
import requests

headers = {
    'Authorization': 'Bearer secret_xxxxxxxxx',
    'Content-Type': 'application/json',
    'Notion-Version': '2026-03-11'
}

response = requests.get(
```

```
'https://api.notion.com/v1/blocks/34051562df03806ab9d7cd40439aef35/child
ren',
    headers=headers
)

print(response.json())
```

7. PATCH /blocks/:id — Update a Block

JavaScript:

```
fetch('https://api.notion.com/v1/blocks/34051562df0380c0bc43fce363bee5bf
', {
  method: 'PATCH',
  headers: {
    'Authorization': 'Bearer secret_xxxxxxxxx',
    'Content-Type': 'application/json',
    'Notion-Version': '2026-03-11'
  },
  body: JSON.stringify({
    paragraph: {
      rich_text: [{ text: { content: 'Updated paragraph text' } }]
    }
  })
})
.then(response => response.json())
.then(data => console.log(data));
```

Python:

```
import requests

headers = {
    'Authorization': 'Bearer secret_xxxxxxxxx',
    'Content-Type': 'application/json',
    'Notion-Version': '2026-03-11'
}

body = {
    'paragraph': {
        'rich_text': [{ 'text': { 'content': 'Updated paragraph text' }
}]
```

```
    }
}

response = requests.patch(
    'https://api.notion.com/v1/blocks/34051562df0380c0bc43fce363bee5bf',
    headers=headers,
    json=body
)

print(response.json())
```

8. DELETE /blocks/:id — Delete a Block

JavaScript:

```
fetch('https://api.notion.com/v1/blocks/34051562df0380c0bc43fce363bee5bf
', {
  method: 'DELETE',
  headers: {
    'Authorization': 'Bearer secret_xxxxxxxxx',
    'Content-Type': 'application/json',
    'Notion-Version': '2026-03-11'
  }
})
.then(response => response.json())
.then(data => console.log(data));
```

Python:

```
import requests

headers = {
    'Authorization': 'Bearer secret_xxxxxxxxx',
    'Content-Type': 'application/json',
    'Notion-Version': '2026-03-11'
}

response = requests.delete(
    'https://api.notion.com/v1/blocks/34051562df0380c0bc43fce363bee5bf',
    headers=headers
)

print(response.json())
```

Changelog

VERSION	DATE	CHANGES
1.0	April 11, 2026	Initial release

Support & Contact

CHANNEL	DETAILS
Official Docs	https://developers.notion.com
Community	https://www.notion.so/community
Status Page	https://www.notion-status.com/

License & Compliance

API Usage

This documentation covers the Notion REST API. Use of the API is subject to Notion's API Terms of Service.

Documentation License

This documentation was created by Gokul S Anand as a technical writing portfolio project. It is intended for educational and demonstration purposes only.

Data Privacy

Never expose your Notion Integration Token (`secret_xxxxxxxxx`) in client-side code or public repositories. Store tokens in environment variables:

JavaScript:

```
const token = process.env.NOTION_API_KEY;
```

Python:

```
import os
token = os.getenv('NOTION_API_KEY')
```

Rate Limiting

Notion's API allows an average of 3 requests per second per integration. Exceeding this limit returns a `429 Too Many Requests` error. Implement exponential backoff in your integration to handle rate limit errors gracefully.

Document Version: 1.0 | Last Reviewed: April 23, 2026